MongoDB Basic Practice Task

1. Database and Collection setup.

```
test> use studentDB
switched to db studentDB
studentDB> db.createCollection("students")
{ ok: 1 }
studentDB> show collections
students
```

2. Insert Operation.

```
studentDB> db.students.insertOne({
|     name: "Sethu",
|     age: 23,
|     course: "MERN Stack",
|     status: "enrolled"
| })
{
  acknowledged: true,
  insertedId: ObjectId('6abcd4b4e4e656ab46fc5641')
}
studentDB> db.students.insertMany([
|     {
|         name: "Arun",
|         age: 22,
|         course: "MERN Stack",
|         status: "enrolled"
|     },
|     {
|         name: "Priya",
|         age: 21,
|         course: "Python Full Stack",
|         status: "completed"
|     },
|     {
|         name: "Rahul",
|         age: 24,
|         course: "MERN Stack",
|         status: "enrolled"
|     },
|     {
|         name: "Divya",
|         age: 20,
|         course: "Data Science",
|         status: "enrolled"
|     }
| ])
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('6abcd51ee4e656ab46fc5642'),
    '1': ObjectId('6abcd51ee4e656ab46fc5643'),
    '2': ObjectId('6abcd51ee4e656ab46fc5644'),
    '3': ObjectId('6abcd51ee4e656ab46fc5645')
  }
}
```

3.

Fetch all the documents from the collection.

```
studentDB> db.students.find()
[
  {
    _id: ObjectId('6abcd6d44c0102a9da49b343'),
    name: 'Sethu',
    age: 23,
    course: 'MERN Stack',
    status: 'enrolled'
  },
  {
    _id: ObjectId('6abcd6e14c0102a9da49b344'),
    name: 'Arun',
    age: 22,
    course: 'MERN Stack',
    status: 'enrolled'
  },
  {
    _id: ObjectId('6abcd6e14c0102a9da49b345'),
    name: 'Priya',
    age: 21,
    course: 'Python Full Stack',
    status: 'completed'
  },
  {
    _id: ObjectId('6abcd6e14c0102a9da49b346'),
    name: 'Rahul',
    age: 24,
    course: 'MERN Stack',
    status: 'enrolled'
  },
  {
    _id: ObjectId('6abcd6e14c0102a9da49b347'),
    name: 'Divya',
    age: 20,
    course: 'Data Science',
    status: 'enrolled'
  }
]
```

Use filters to fetch specific data

```
studentDB> db.students.find({
|     course: "MERN Stack"
| })
[
  {
    _id: ObjectId('6abcd6d44c0102a9da49b343'),
    name: 'Sethu',
    age: 23,
    course: 'MERN Stack',
    status: 'enrolled'
  },
  {
    _id: ObjectId('6abcd6e14c0102a9da49b344'),
    name: 'Arun',
    age: 22,
    course: 'MERN Stack',
    status: 'enrolled'
  },
  {
    _id: ObjectId('6abcd6e14c0102a9da49b346'),
    name: 'Rahul',
    age: 24,
    course: 'MERN Stack',
    status: 'enrolled'
  }
]
```

4.

Update a single document

```
studentDB> db.students.updateOne(
|     { name: "Sethu" },
|     { $set: { status: "completed" } }
| )
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
studentDB> db.students.find({ name: "Sethu" })
[
  {
    _id: ObjectId('6abcd6d44c0102a9da49b343'),
    name: 'Sethu',
    age: 23,
    course: 'MERN Stack',
    status: 'completed'
  }
]
```

Update multiple documents at once.

```
studentDB> db.students.updateMany(
|     {
|         course: "MERN Stack",
|         status: "enrolled"
|     },
|     {
|         $set: { status: "active" }
|     }
| )
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 2,
  modifiedCount: 2,
  upsertedCount: 0
}
studentDB> db.students.find({ course: "MERN Stack" })
[
  {
    _id: ObjectId('6abcd6d44c0102a9da49b343'),
    name: 'Sethu',
    age: 23,
    course: 'MERN Stack',
    status: 'completed'
  },
  {
    _id: ObjectId('6abcd6e14c0102a9da49b344'),
    name: 'Arun',
    age: 22,
    course: 'MERN Stack',
    status: 'active'
  },
  {
    _id: ObjectId('6abcd6e14c0102a9da49b346'),
    name: 'Rahul',
    age: 24,
    course: 'MERN Stack',
    status: 'active'
  }
]
```

5. Delete Operations

```
studentDB> db.students.deleteOne({
|     name: "Divya"
| })
{ acknowledged: true, deletedCount: 1 }
studentDB> db.students.find()
[
  {
    _id: ObjectId('6abcd6d44c0102a9da49b343'),
    name: 'Sethu',
    age: 23,
    course: 'MERN Stack',
    status: 'completed'
  },
  {
    _id: ObjectId('6abcd6e14c0102a9da49b344'),
    name: 'Arun',
    age: 22,
    course: 'MERN Stack',
    status: 'active'
  },
  {
    _id: ObjectId('6abcd6e14c0102a9da49b345'),
    name: 'Priya',
    age: 21,
    course: 'Python Full Stack',
    status: 'completed'
  },
  {
    _id: ObjectId('6abcd6e14c0102a9da49b346'),
    name: 'Rahul',
    age: 24,
    course: 'MERN Stack',
    status: 'active'
  }
]
studentDB> db.students.deleteMany({})
{ acknowledged: true, deletedCount: 4 }
studentDB> db.students.find()

studentDB>
```

6.

Query Operators- $and



Query Operators- $exists

```
studentDB> db.students.find({
|     course: { $exists: true }
| })
[
  {
    _id: ObjectId('6abcd6d44c0102a9da49b343'),
    name: 'Sethu',
    age: 23,
    course: 'MERN Stack',
    status: 'completed'
  },
  {
    _id: ObjectId('6abcd6e14c0102a9da49b344'),
    name: 'Arun',
    age: 22,
    course: 'MERN Stack',
    status: 'active'
  },
  {
    _id: ObjectId('6abcd6e14c0102a9da49b345'),
    name: 'Priya',
    age: 21,
    course: 'Python Full Stack',
    status: 'completed'
  },
  {
    _id: ObjectId('6abcd6e14c0102a9da49b346'),
    name: 'Rahul',
    age: 24,
    course: 'MERN Stack',
    status: 'active'
  },
  {
    _id: ObjectId('6abcd6e14c0102a9da49b347'),
    name: 'Divya',
    age: 20,
    course: 'Data Science',
    status: 'enrolled'
  }
]
```

Query Operators- $gt

```
studentDB> db.students.find({
|     age: { $gt: 22 }
| })
[
  {
    _id: ObjectId('6abcd6d44c0102a9da49b343'),
    name: 'Sethu',
    age: 23,
    course: 'MERN Stack',
    status: 'completed'
  },
  {
    _id: ObjectId('6abcd6e14c0102a9da49b346'),
    name: 'Rahul',
    age: 24,
    course: 'MERN Stack',
    status: 'active'
  }
]
```

Query Operators- $in

```
studentDB> db.students.find({
|     course: {
|         $in: ["MERN Stack", "Data Science"]
|     }
| })
[
  {
    _id: ObjectId('6abcd6d44c0102a9da49b343'),
    name: 'Sethu',
    age: 23,
    course: 'MERN Stack',
    status: 'completed'
  },
  {
    _id: ObjectId('6abcd6e14c0102a9da49b344'),
    name: 'Arun',
    age: 22,
    course: 'MERN Stack',
    status: 'active'
  },
  {
    _id: ObjectId('6abcd6e14c0102a9da49b346'),
    name: 'Rahul',
    age: 24,
    course: 'MERN Stack',
    status: 'active'
  },
  {
    _id: ObjectId('6abcd6e14c0102a9da49b347'),
    name: 'Divya',
    age: 20,
    course: 'Data Science',
    status: 'enrolled'
  }
]
```

Query Operators- $lt

```
studentDB> db.students.find({
|     age: { $lt: 22 }
| })
[
  {
    _id: ObjectId('6abcd6e14c0102a9da49b345'),
    name: 'Priya',
    age: 21,
    course: 'Python Full Stack',
    status: 'completed'
  },
  {
    _id: ObjectId('6abcd6e14c0102a9da49b347'),
    name: 'Divya',
    age: 20,
    course: 'Data Science',
    status: 'enrolled'
  }
]
```

Query Operators- $or

```
studentDB> db.students.find({
|     $or: [
|         { course: "MERN Stack" },
|         { course: "Data Science" }
|     ]
| })
[
  {
    _id: ObjectId('6abcd6d44c0102a9da49b343'),
    name: 'Sethu',
    age: 23,
    course: 'MERN Stack',
    status: 'completed'
  },
  {
    _id: ObjectId('6abcd6e14c0102a9da49b344'),
    name: 'Arun',
    age: 22,
    course: 'MERN Stack',
    status: 'active'
  },
  {
    _id: ObjectId('6abcd6e14c0102a9da49b346'),
    name: 'Rahul',
    age: 24,
    course: 'MERN Stack',
    status: 'active'
  },
  {
    _id: ObjectId('6abcd6e14c0102a9da49b347'),
    name: 'Divya',
    age: 20,
    course: 'Data Science',
    status: 'enrolled'
  }
]
```

7. Create a small use case like a Library System or E-Commerce product listing and apply insert, update, and search operations.

Insert Operation

```
studentDB> use libraryDB
switched to db libraryDB
libraryDB> db.createCollection("books")
{ ok: 1 }
libraryDB> db.books.insertMany([
|     {
|         title: "The Alchemist",
|         author: "Paulo Coelho",
|         category: "Fiction",
|         price: 350,
|         available: true
|     },
|     {
|         title: "Clean Code",
|         author: "Robert C. Martin",
|         category: "Programming",
|         price: 550,
|         available: true
|     },
|     {
|         title: "JavaScript: The Good Parts",
|         author: "Douglas Crockford",
|         category: "Programming",
|         price: 450,
|         available: false
|     },
|     {
|         title: "Atomic Habits",
|         author: "James Clear",
|         category: "Self Help",
|         price: 500,
|         available: true
|     }
| ])
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('6abce18c4c0102a9da49b348'),
    '1': ObjectId('6abce18c4c0102a9da49b349'),
    '2': ObjectId('6abce18c4c0102a9da49b34a'),
    '3': ObjectId('6abce18c4c0102a9da49b34b')
  }
}
```

Insert Operation

```
libraryDB> db.books.find()
[
  {
    _id: ObjectId('6abce18c4c0102a9da49b348'),
    title: 'The Alchemist',
    author: 'Paulo Coelho',
    category: 'Fiction',
    price: 350,
    available: true
  },
  {
    _id: ObjectId('6abce18c4c0102a9da49b349'),
    title: 'Clean Code',
    author: 'Robert C. Martin',
    category: 'Programming',
    price: 550,
    available: true
  },
  {
    _id: ObjectId('6abce18c4c0102a9da49b34a'),
    title: 'JavaScript: The Good Parts',
    author: 'Douglas Crockford',
    category: 'Programming',
    price: 450,
    available: false
  },
  {
    _id: ObjectId('6abce18c4c0102a9da49b34b'),
    title: 'Atomic Habits',
    author: 'James Clear',
    category: 'Self Help',
    price: 500,
    available: true
  }
]
```

Search Operations

```
libraryDB> db.books.find({
|     category: "Programming"
| })
[
  {
    _id: ObjectId('6abce18c4c0102a9da49b349'),
    title: 'Clean Code',
    author: 'Robert C. Martin',
    category: 'Programming',
    price: 550,
    available: true
  },
  {
    _id: ObjectId('6abce18c4c0102a9da49b34a'),
    title: 'JavaScript: The Good Parts',
    author: 'Douglas Crockford',
    category: 'Programming',
    price: 450,
    available: false
  }
]
libraryDB> db.books.find({
|     price: { $lt: 500 }
| })
[
  {
    _id: ObjectId('6abce18c4c0102a9da49b348'),
    title: 'The Alchemist',
    author: 'Paulo Coelho',
    category: 'Fiction',
    price: 350,
    available: true
  },
  {
    _id: ObjectId('6abce18c4c0102a9da49b34a'),
    title: 'JavaScript: The Good Parts',
    author: 'Douglas Crockford',
    category: 'Programming',
    price: 450,
    available: false
  }
]
```

Search Operations



Update Operation

```
libraryDB> db.books.updateOne(
|     { title: "JavaScript: The Good Parts" },
|     { $set: { available: true } }
| )
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 0,
  upsertedCount: 0
}
libraryDB> db.books.find({
|     available: true
| })
[
  {
    _id: ObjectId('6abce18c4c0102a9da49b348'),
    title: 'The Alchemist',
    author: 'Paulo Coelho',
    category: 'Fiction',
    price: 350,
    available: true
  },
  {
    _id: ObjectId('6abce18c4c0102a9da49b349'),
    title: 'Clean Code',
    author: 'Robert C. Martin',
    category: 'Programming',
    price: 550,
    available: true
  },
  {
    _id: ObjectId('6abce18c4c0102a9da49b34a'),
    title: 'JavaScript: The Good Parts',
    author: 'Douglas Crockford',
    category: 'Programming',
    price: 450,
    available: true
  },
  {
    _id: ObjectId('6abce18c4c0102a9da49b34b'),
    title: 'Atomic Habits',
    author: 'James Clear',
    category: 'Self Help',
    price: 500,
    available: true
  }
]
```

Update Operation



```
libraryDB> db.books.find({
|     price: { $gt: 400 }
| })
[
  {
    _id: ObjectId('6abce18c4c0102a9da49b349'),
    title: 'Clean Code',
    author: 'Robert C. Martin',
    category: 'Programming',
    price: 550,
    available: true
  },
  {
    _id: ObjectId('6abce18c4c0102a9da49b34a'),
    title: 'JavaScript: The Good Parts',
    author: 'Douglas Crockford',
    category: 'Programming',
    price: 450,
    available: true
  },
  {
    _id: ObjectId('6abce18c4c0102a9da49b34b'),
    title: 'Atomic Habits',
    author: 'James Clear',
    category: 'Self Help',
    price: 500,
    available: true
  }
]
```